

Dynamic Time Window based Reward for Reinforcement Learning in Continuous Integration Testing

Chaoyue Pan

Beijing University of Chemical Technology
Chaoyang District, Beijing, China
pcy030@qq.com

Zheng Li*

Beijing University of Chemical Technology
Chaoyang District, Beijing, China
lizheng@mail.buct.edu.cn

Yang Yang

Beijing University of Chemical Technology
Chaoyang District, Beijing, China
yyyilian8695@163.com

Junxia Guo

Beijing University of Chemical Technology
Chaoyang District, Beijing, China
gjxia@mail.buct.edu.cn

ABSTRACT

Continuous Integration (CI) testing is an expensive, time-consuming, and resource-intensive process. Test case prioritization (TCP) can effectively reduce the workload of regression testing in the CI environment, where Reinforcement Learning (RL) is adopted to prioritize test cases, since the TCP in CI testing can be formulated as a sequential decision-making problem, which can be solved by RL effectively. A useful reward function is a crucial component in the construction of the CI system and a critical factor in determining RL's learning performance in CI testing. This paper focused on the validity of the execution history information of the test cases on the TCP performance in the existing CI testing optimization methods based on RL, and a Dynamic Time Window based reward function are proposed by using partial information dynamically for fast feedback and cost reduction. Experimental studies are carried out on six industrial datasets. The experimental results showed that using dynamic time window based reward function can significantly improve the learning efficiency of RL and the fault detection ability when comparing with the reward function based on fixed time window.

KEYWORDS

continuous integration, reward function, dynamic time window, test case prioritization, reinforcement learning

ACM Reference Format:

Chaoyue Pan, Yang Yang, Zheng Li, and Junxia Guo. 2020. Dynamic Time Window based Reward for Reinforcement Learning in Continuous Integration Testing. In *12th Asia-Pacific Symposium on Internetware (Internetware'20)*, May 12–14, 2021, Singapore, Singapore. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3457913.3457930>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Internetware'20, May 12–14, 2021, Singapore, Singapore
© 2020 Association for Computing Machinery.
ACM ISBN 978-1-4503-8819-1/20/11...\$15.00
<https://doi.org/10.1145/3457913.3457930>

1 INTRODUCTION

Continuous Integration (CI) testing requires team members to integrate their work frequently, usually at least once a day for each person, indicating that software development requires multiple integrations every day¹. Each integration is validated by an automated build and regression testing to detect faults as quickly as possible [10, 24]. Because of the daily need for frequent integration tests, it becomes challenging to ensure that the test schedule is consistent with the development schedule.

Regression testing is a critical component of CI testing [32]. Selecting and prioritizing test cases that potentially detect errors will effectively improve the efficiency of CI regression testing and then speed up the team's testing progress. Common solutions include test case selection (TCS) [25, 29, 36], test suite minimization (TSM) [23, 28], and test case prioritization (TCP) [2, 13, 22, 26]. TCS identifies parts of the program that need to be retested or have been modified, and selects test cases need to be executed accordingly aiming to reduce the overhead of regression testing. TSM minimizes the test case set by removing the test cases from the test case set. TCP is a technique of prioritizing test cases before they are executed to find failed test cases as early as possible. TCS and TCP are the two main techniques used in CI testing optimization. D. Marijan et al. [3] studied the TCP for regression testing in CI and used the historical execution results of test cases to determine the optimal sequencing of subsequent test cases and achieved good results.

In recent years, many researchers are trying to use machine learning to prioritize test cases. Spieker et al. [30] first combined reinforcement learning (RL) with TCP problem in CI testing and proposed a test case selection and prioritization method based on RL and artificial neural network (RETECS).

The core of RL is to study the process of interaction with the environment. In other words, it decides the current action based on the historical interaction information, and its essence is to solve the sequential decision-making problems and can be used for CI testing optimization. RL uses the reward function to calculate the instant reward after the agent selects an action, which is the direct feedback of the agent's interaction with the environment. The use of reward function in RL plays a crucial role in the efficiency of TCP in CI testing [35]. RL algorithm obtains the state-to-behavior mapping

¹http://wikipedia.moesalih.com/Continuous_integration

through the continuous interaction between the agent and the environment. A useful reward function can more accurately reflect the interaction between the two so that the agent can perceive its environment more accurately, thereby speeding up the convergence process of RL. Therefore, designing or improving a useful reward function builds the most critical and extremely difficult part of the RL system. It is also a critical factor determining whether the RL system can have a good learning effect.

In the RETECS method proposed by Spieker, the reward function only considers the test case execution information of the current cycle. It lacks consideration of the overall execution history of the test case. Yang et al. [35] proposed a new reward function based on the complete test case history execution information. The reward function effectively improves the fault detection ability of the CI test sequence. However, in the CI environment, due to the limited test time, the factors that affect the effectiveness of test prioritization mainly include the size of the test suite and the test execution history [21]. Because of large amount of CI cycles and the limited time of CI testing, it is not advisable to use all test execution history to determine the test priority sequence. To determine the priority of the test, we must first determine how much execution history to use. If only using the most recent execution history, it is possible to miss more essential test cases. Furthermore, the ancient execution histories are not necessary to be used because these histories may contain much-outdated information, leading to the generated test sequence's low fault detection capabilities. Reasonable use of the execution history information of test cases can effectively improve the performance of TCP in regression testing [21]. Wu et al. [34] applied the concept of time window to CI test optimization based on RL and proposed a reward function based on fixed time window. It solved the problem of redundancy in historical information of test cases, and at the same time effectively improved the fault detection ability of the prioritized test sequence. However, with the increasing CI cycles, the historical information of test cases is continually changing, the window size of the reward function using fixed time window cannot be adjusted accordingly to changes in the CI environment, that is, it is not adaptive. Therefore, this paper researches the reward function design, comprehensively considers the historical failure interval of test cases in CI cycle, defines a dynamic time window, and proposes a dynamic time window based reward function which is adaptive to the CI environment.

The main contributions of this paper include:

- Aiming at the time limit of regression testing in the CI environment, the definition of dynamic time window is proposed.
- In the RL reward function design for CI testing optimization, the reward function based on dynamic time window is proposed.
- An empirical study is carried out on six actual industrial subjects, and the results show that in the optimization of software CI testing, the proposed novel reward function based on the dynamic time window is more effective than the reward function based on fixed time window to prioritize test cases, and further improves the learning efficiency of RL.

This paper is organized as follows: Section 2 summarizes the related work and leads to the motivation of this paper. Section 3

introduces the dynamic time window based reward function in detail. Section 4 empirically verifies the validity of the rewards proposed in this paper, while Section 5 introduces the threats to validity. Section 6 concludes our work and describe our future work.

2 BACKGROUND AND RELATED WORK

The Test Case Prioritization (TCP) technique aims to find the best and orderly sequence of test cases to detect failures in the regression testing process as early as possible. Testers hope that test cases with strong fault detection capabilities are executed to detect failures as early as possible and reduce regression testing loss. The TCP technique was first proposed by Wong et al. [33]. They searched for test cases in the regression test process to find the optimal test case execution sequence that satisfies the test criteria. With the development of the times, many TCP techniques have emerged. The existing prioritization methods for test cases include test case coverage information based [6], program modification information based [14], fault-based [19], requirement-based [31] and history-based [7].

CI is an efficient software development practice. The purpose of CI is to allow products to be iterated quickly while maintaining high quality. It requires team members to integrate their code into the backbone through automatic regression testing frequently. However, due to the need for fast feedback, the time for automatic regression testing is limited.

In 2011, Hyunsook Do et al. [9] evaluated the cost and benefit impact of time constraints on prioritizing test cases through experiments and found that time constraints played an essential role in the cost-effectiveness of prioritization and the relative cost-effectiveness trade-off between technology. In 2014, Elbaum et al. [11] proposed the concept of a time window (Test Suite Execution Windows) to track the recent execution history of the test suite. In 2016, Marijan et al. [21] analyzed the impact of historical time windows on the fault detection effect of prioritized test suite in regression testing, and through a series of experiments, proved that the size of the time window has a significant impact on the fault detection performance of regression testing. In 2017, Kim et al. [15] proposed an FHD algorithm based on historical failure data. It can analyze test case failure historical data in real-time. During CI testing, if an error occurs in a test case in the current cycle, FHD will prioritize test cases effectively according to the relevant test case historical data.

The concept of the dynamic time window is widely used in many fields. Shao et al. [27] adopted sliding time window to model the sync constraints in city logistics for the first time, significantly improving vehicle travel efficiency. Deng et al. [8] proposed a sliding-time window (STW) based features extraction algorithm (FEA). This algorithm can extract the MMC features based on the feature relationship between neighboring STWs to locate the fault with short time and high accuracy for the MMC. In the area of human activity recognition, Ma et al. [18] proposed an adaptive time window-based algorithm of activity recognition. They use the time window expansion and contraction method to determine and dynamically adjust the window size for each activity, which significantly improves the algorithm's performance.

The TCP problem in CI is essentially similar to a sequential decision-making problem, and RL is suitable for solving this type of problem. When the agent interacts with the environment through actions, the environment will return the agent an instant reward of the current action. The agent evaluates the actions taken according to the current reward value, that is, actions that are beneficial to achieving the goal are retained, and actions that are not conducive to achieving the goal are attenuated. Busjaeger et al. [4] applied machine learning and a variety of heuristic learning techniques to prioritize test cases. Chen et al. [5] applied the idea of semi-supervised clustering to the selection of regression tests, but this method has the problem of high computational complexity. In 2017, Spieker et al. combined RL with prioritization of test cases in CI for the first time. They proposed a new lightweight test case selection and prioritization method (RETECS) based on RL and artificial neural networks. Figure 1 shows the testing process in RETECS method. The RETECS method uses RL to select and prioritize test cases based on their duration, last execution, and failure history. It evaluates and learns the previously generated test sequence according to the test case's execution result after each CI cycle and rewards the test case's priority that needs to be executed in the next cycle through the reward function.

RL uses a reward function to calculate the reward value of the agent's current action. This is the direct feedback of the agent's interaction with the environment. A useful reward function is a vital component in building a RL system. The critical factor that determines whether the RL system can have a good learning effect. In 2018, He et al. [17] analyzed the reward function's defects in the RETECS method. They studied the impact of the historical failure distribution of test cases on TCP performance, and proposed a novel reward function based on the average percentage of historical failure, named APHF. The experimental results showed that the test sequence's fault detection ability generated by APHF reward is better than the reward function effect of the RETECS method. In 2019, Wu et al. [34] proposed a reward function using a fixed time window, named APHFW, which is more effective to prioritize test cases than APHF. They found that APHFW performed best when the time window was set to 5.

3 REWARD FOR REINFORCEMENT LEARNING IN CI TESTING

Many studies have shown that test optimization based on time windows has good performance. In this section, we first introduce the definition of the dynamic time window in detail, and then propose the reward function based on dynamic time window.

3.1 Dynamic Time Window Setting

A test case will be executed many times in whole CI testing process. We assume that HR is a list representing the historical execution result of a test case, i represents the CI cycle. For a test case t , $HR_i(t) = [r_n, \dots, r_2, r_1]$ represents the execution result of n times of test case t from the first integration to the i^{th} integration process. There are two values for r_n , 0 and 1. 0 means that the test case t passed the n^{th} execution and 1 means that the test case t failed the n^{th} execution.

We assume that r_n and r_2 are both 1, that is, the test case t fails in the second and n^{th} executions, and after t fails in the second execution and before the n^{th} execution fails, its execution results are both pass status. We call the distance between r_n and r_2 a historical failure interval I of test case t .

Similarly, for a test case t , we use $HFI_i(t) = [I_n, \dots, I_2, I_1]$ to represent the historical failure interval list of length n from the first integration to the i^{th} integration.

In the CI testing environment, many test cases need to be executed for each integration, and test cases that need to be executed in each cycle are continually changing. Some test cases may be executed once but not executed or rarely executed in subsequent cycles. Then, the information of these test cases is out of date for the test optimization of the next cycle. Considering these relevant information will not help the test optimization of the current cycle and may even reduce the fault detection capability of the generated test sequence. Therefore, it is not appropriate to consider the whole history information. We further analyzed this finding to explore how to use test history data optimally to improve the performance of CI testing.

The average historical failure interval of a test case reflects the historical average failure distribution of the test case. For test cases that often fail, the average failure interval is relatively small. For test cases that accidentally fail, the average failure interval is relatively large. Observed from the time dimension of test cases, for the recent execution of the test cases, we use the average historical failure interval of test cases as the time window to select part of the historical information, if the failures are more intensive within the average level, these test cases are more likely to fail in the next integration cycle or frequently fail in subsequent integration cycles. On the contrary, if there are fewer failures within the recent average level, the fault detection ability of these test cases will be reduced accordingly.

Therefore, we comprehensively considered the average historical failure interval of the test case sequence in each CI cycle, and effectively combined it with the concept of time window, and proposed the definition of dynamic time window, that is, a dynamic time interval, which captures the test execution history used to optimize the test sequence in subsequent CI testing. The definition of dynamic time window is as follows:

Definition 3.1. Dynamic Time Window (DTW)

$$DTW_i = \frac{\sum_{t \in TS_i} \frac{\sum_{l \in HFI_i(t)} I}{|HFI_i(t)|}}{|TS_i|} \quad (1)$$

DTW_i represents the dynamic time window used in i^{th} CI cycle, TS_i represents the test sequence of the i^{th} cycle, $|TS_i|$ represents the total number of test cases in TS_i .

3.2 The Reward Function based on Dynamic Time Window

The Average Percentage of Historical Failure (APHF) is defined to measure a test case's historical failure distribution. Based on this, the APHF reward function was proposed [17]. A test case's historical information may contain a lot of useless and outdated information, which will affect the test sequence's fault detection

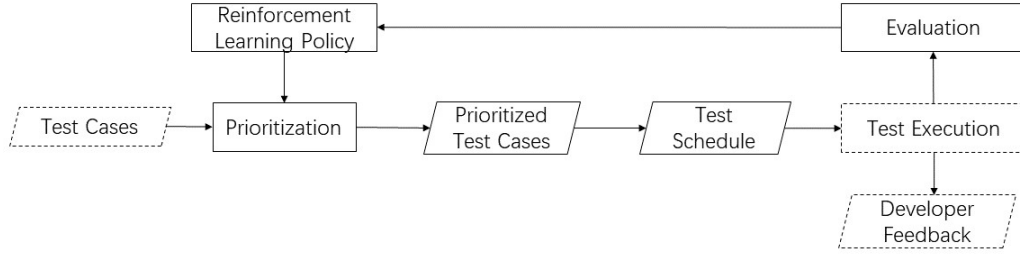


Figure 1: Testing in CI process: RETECS uses test execution results for learning test case prioritization (solid boxes: Included in RETECS, dashed boxes: Interfaces to the CI environment)

ability. For example, one test case detects an error in the early stage of CI, but in the subsequent consecutive cycles of long time, it does not detect any errors. Then, the execution information of the test case during these cycles is of little value to test optimization, and these information may be useless. We do not have to waste computing resources to consider this piece of historical information. Therefore, we use a dynamic time window to capture partial historical execution information of test case to improve the performance of CI testing. Based on the definition of APHF reward, we further propose a dynamic time window based reward function named APHF_DTW, to optimize the CI testing optimization framework based on RL. APHF_DTW is defined as follows:

Definition 3.2. Average Percentage of Historical Failure Reward with DTW (APHF_DTW)

$$APHF_DTW_i(t) = 1 - \frac{\sum_{j=1}^m R_j}{dtw(DTW_i, n) \times m} + \frac{1}{2 \times dtw(DTW_i, n)} \quad (2)$$

Where i represents the i^{th} CI cycle, n represents the number of historical execution results of a test case t from the first integration cycle to the i^{th} cycle. DTW_i represents the dynamic time window used in the i^{th} cycle, and $dtw(DTW_i, n)$ represents the number of historical execution results used by t in the i^{th} cycle. If $DTW_i < n$, then the historical execution result time window used by t in the i^{th} cycle is DTW_i , otherwise, take n . m represents the number of execution failures of t within the time window of the historical execution result determined from $dtw(DTW_i, n)$. R_j represents the countdown order of the recent j^{th} failure of the test case in the history execution, $APHF_DTW \in (0, 1)$.

The greater the value of APHF_DTW is, the greater the probability that test case will fail in the most recent execution cycle is.

Figure 2 is the framework diagram of the RL method using APHF_DTW in CI. The agent selects the test sequence and sends it to the CI environment. The CI environment generates a dynamic time window for current cycle based on the current cycle's test case execution information. APHF_DTW selects part of the test case's historical information based on the range obtained by the dynamic time window, calculates the reward, and rewards the test case. The reward information is feedback to the agent for learning.

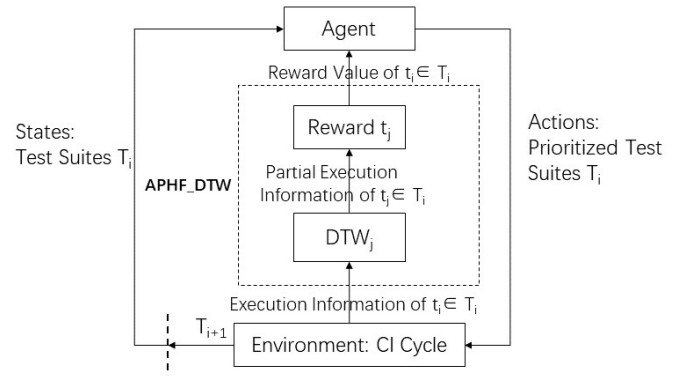


Figure 2: The framework of reinforcement learning with dynamic time window based reward for CI testing

The agent learns how to optimally sort the test cases in this process to maximize the test sequence's fault detection ability.

4 EXPERIMENTAL EVALUATION

In order to verify the effectiveness of APHF_DTW proposed for the optimization and sorting technology of test cases based on RL, this paper conducts experiments and analysis on the following three research questions:

- RQ1:** How does the reward function based on dynamic time window affect the learning efficiency of RL compared with the reward function based on fixed time window?
- RQ2:** Is the reward function based on dynamic time window more effective to prioritize test cases than the existing reward function based on fixed time window?
- RQ3:** Does the reward function based on dynamic time window affect the overall efficiency of CI testing in terms of time cost?

This paper proposes three research questions. The purpose of RQ1 is to verify the effectiveness of the reward function based on dynamic time window for the learning efficiency of RL. In order to answer RQ1, it is necessary to compare the periodic trend of the fault detection ability of the test sequence generated by the RL method with dynamic time window based reward function and the test sequence generated by the reward function based on fixed time

window, that is to compare the pros and cons of APHF_DTW and APHFW. RQ2 is to verify whether our method can help improve the fault detection ability of the generated test sequence. In order to answer RQ2, we compared the pros and cons of the three indicators of NAPFD, Recall and TTF generated by APHF_DTW and APHFW. RQ3 compares the efficiency of the RL framework under the guidance of APHF_DTW and APHFW.

In this section, the evaluation metric used in the experiments is introduced in Section 4.1 and then the experimental setup is in Section 4.2. Finally, Section 4.3 analyzes the experimental results.

4.1 Evaluation Metric

4.1.1 NAPFD. In this paper, NAPFD (Normalized Average Percentage of Faults Detected) [20] is adopted as the first evaluation metric, which is defined as follows:

$$NAPFD(TS_i) = p - \frac{\sum_{t \in TS_i^{fail}} rank(t)}{|TS_i^{fail}| \times |TS_i|} + \frac{p}{2 \times |TS_i|} \quad (3)$$

$$\text{with } p = \frac{|TS_i^{fail}|}{|TS_i^{total, fail}|}$$

Where TS_i represents the test sequence that needs to be executed in a test schedule in i^{th} CI cycle, $|TS_i|$ represents the total number of test cases in TS_i , $|TS_i^{fail}|$ represents the total number of failed test cases in TS_i , $|TS_i^{total, fail}|$ represents the total number of failed test cases in test case set in i^{th} CI cycle, and $rank(t)$ represents the position of the t^{th} failed test case in TS_i .

In 1999, Rothermel et al. [11] first proposed the average percentage of faults detected (APFD) to measure the effectiveness of the TCP technique. APFD is suitable for situations where the entire test suite can be run and all faults can be found. However, regression testing in CI usually only executes part of the test suite due to time limit of CI. NAPFD is an extension of APFD, which can reflect the proportion of errors detected in test suite, and is suitable for TCS in CI. If all errors are detected, NAPFD is same as APFD ($p = 1$). When calculating the NAPFD value in this experiment, we assume that the error corresponding to each failure is different.

4.1.2 RECALL. Recall indicates the percentage of the number of test cases detected failure in the total number of all failed test cases [35]. The higher Recall rate is, the more failed test cases are detected by test sequence, and the stronger fault detection ability of test sequence is.

4.1.3 TTF. Test To Failure (TTF) represents the position where the first detected test case fails [35]. It is an important metric in TCP. The smaller the TTF value is, the earlier the test sequence finds errors, and the stronger the fault detection ability of the test sequence is.

4.2 Experiment Setup

In our experiment, we use the source program provided by Spieker et al. [30] and reproduce the reward function based on fixed time window. On the basis of the tuning experiment, we set the fixed time window size to 5 [34]. For the range setting of dynamic time

window in our experiment, if the window setting is too small, the selected information is close to the latest execution result, or if it's too large, and the selected information is close to whole execution history. In order to avoid this situation, we set the size range of the dynamic time window between 3 and 10 based on literature research [21] [15] [34] [20] [12]. Based on the source program, we used the reward function designed by us to carry out experiments.

In order to determine the applicability of the method in the actual industry, we used three industrial datasets provided by the original author [30], and on this basis, we expanded three industrial datasets, using a total of six industrial datasets. *Paint Control* and *IOF/ROL* are from ABB Robotics Norway, used to test complex industrial robots; *Google Shared Dataset of Test Suite Results* (GSDTSR) is Google's open source dataset. The above three datasets are from the original author. The fourth is *Rails* which comes from a large Ruby-based open source platform. *Mybatis* and *Apache Drill* are extracted from Travis Torrent (a free database based on Travis CI and GitHub) [1]. Table 1 lists the detailed information of the six industrial datasets used in this paper. The first column is the name of the industrial dataset. The second column 'Test cases' is the number of test cases used for testing in the dataset. The third column 'CI cycles' is the number of cycles included in the dataset. The fourth column 'Results' indicates the total number of executions of all test cases in all cycles of the entire dataset. The fifth column 'Failure rate' is the failure rate of the test cases of the entire dataset, and the sixth column 'Frequency' is the frequency of each test case in each cycle. As can be seen from Table 1, the data scale of Paint Control and IOF/ROL is small, but its test case failure rate is much higher than that of GSDTSR and Rails. Paint Control is close to 20%, and IOF/ROL is close to 30%. Google's open source dataset GSDTSR, large industrial-grade dataset Rails and Travis Torrent's dataset Mybatis have large data scales, but the failure rate is less than 1%, respectively 0.25%, 0.62%, 0.15%, that is, there are only a small number of failure test cases in a large number of test cases.

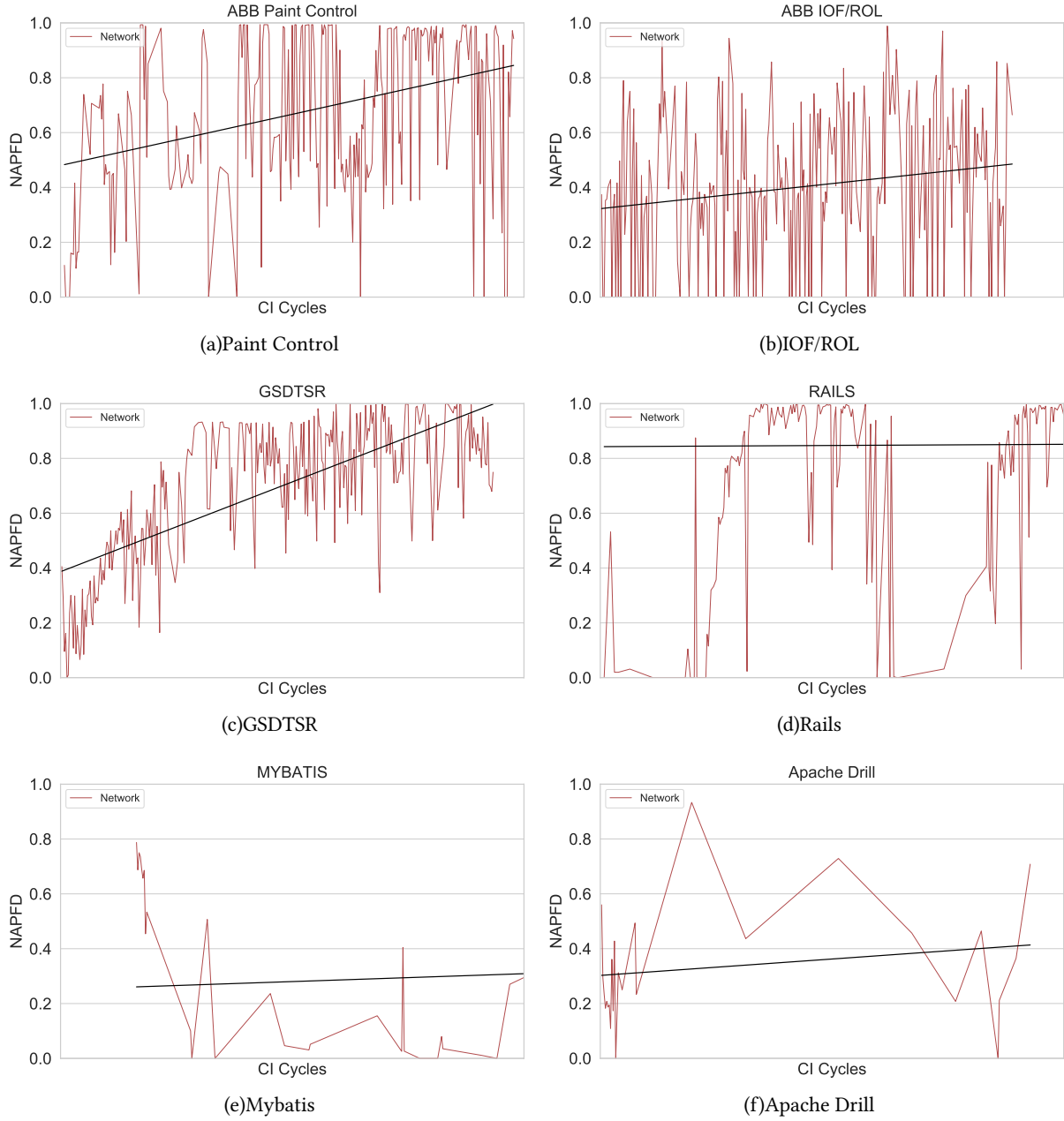
4.3 Experimental Results and Analysis

RQ1: How does the reward function based on dynamic time window affect the learning efficiency of RL compared with the reward function based on fixed time window?

Figure 3 and Figure 4 presents the NAPFD results of two reward functions. The abscissa represents the number of integration cycles, and the ordinate represents the NAPFD value of the test sequence. The black line is fitted according to the NAPFD value, and it represents the learning trend of RL under different reward functions. It can be seen from Figure 3 and Figure 4 that on IOF/ROL and Drill datasets, the slope of the APHF_DTW line is significantly higher than the slope of the APHFW line, indicating that APHF_DTW has a better effect on the learning efficiency of RL on these two datasets. On GSDTSR, the starting point of the APHF_DTW reward line is significantly higher than APHFW. Although the end point of APHF_DTW is not as high as APHFW, it can be clearly seen from the figure that most of the APHF_DTW's NAPFD values are distributed significantly higher than the APHFW's NAPFD values distribution position, indicating that APHF_DTW is also better than APHFW for the learning efficiency of RL on GSDTSR. The situation of Rails dataset is complicated due to its low failure rate,

Table 1: Overview of the Experimental Datasets

Dataset	Test cases	CI cycles	Results	Failure rate	Frequency
Paint Control	89	352	25,594	19.36%	0.82
IOF/ROL	1,941	320	32,260	28.79%	0.05
GSDTSR	5,555	336	1,260,618	0.25%	0.68
Rails	2,010	3,263	781,273	0.62%	0.12
Mybatis	303	988	815,598	0.15%	2.72
Apache Drill	556	376	8,887	2.28%	0.05

**Figure 3: The NAPFD results of APHFW reward**

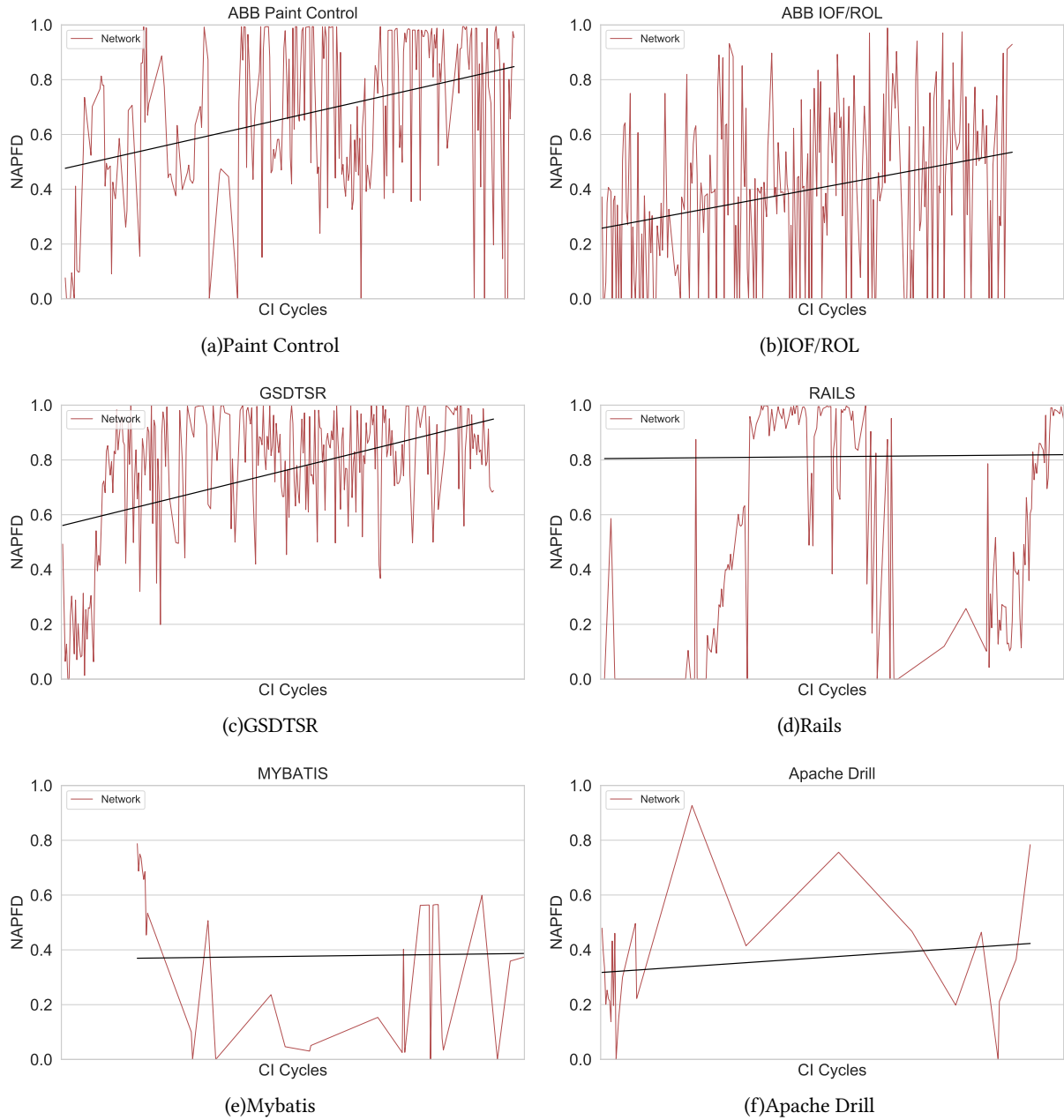


Figure 4: The NAPFD results of APHF_DTW reward

so APHF_DTW is slightly worse than APHFW on this dataset. On Mybatis, the straight line starting point and focus of APHF_DTW are significantly higher than APHFW, indicating that APHF_DTW is better than APHFW for the learning effect of RL on this dataset. On Apache Drill, the slope of the learning trend line of APHF_DTW is higher than APHFW, so on this dataset, APHF_DTW is better than APHFW.

Regarding RQ1, we conclude that the learning efficiency of the reward function based on dynamic time window is better than the

reward function based on fixed time window on most datasets. In other words, it can achieve better learning effect under the framework of RL, and better TCP effect will be obtained as CI progresses.

RQ2: Is the reward function based on dynamic time window more effective to prioritize test cases than the reward function based on fixed time window?

In order to illustrate the effectiveness of the reward function based on dynamic time window to prioritize test cases, we compare

Table 2: The Average NAPFD, Recall, TTF Value of APHF_DTW and APHFW

Dataset	APHFW			APHF_DTW		
	NAPFD	Recall	TTF	NAPFD	Recall	TTF
Paint Control	72.42%	83.24%	2.8	72.56%	83.31%	2.56
IOF/ROL	43.10%	52.9%	1.74	44.42%	54.38%	1.2
GSDTSR	81.42%	85.37%	12.73	82.74%	87.01%	2.81
Rails	89.75%	93.18%	3.65	89.73%	93.08%	3.8
Mybatis	38.79%	90.05%	356.91	41.43%	91.65%	336.14
Apache Drill	39.76%	61.82%	13.58	40.68%	61.62%	13.11

the fault detection capabilities of APHF_DTW and APHFW, and we use NAPFD, Recall and TTF.

Table 2 lists the average NAPFD, Recall and TTF value between two reward functions. The bold value in the table indicates that the value is better. For NAPFD metric, compared to APHFW, APHF_DTW can increase by 2.64% at the maximum and 0.14% at the minimum. The NAPFD value of APHF_DTW is higher than APHFW on the other five datasets except for Rails. On Paint Control, the value increases by 0.12%. On IOF/FOL and GSDTSR, the value all increases by 1.32%. On Rails, it decreases by 0.02%. On Mybatis, the value increases by 2.64%. On Apache Drill, the value increases by 0.92%.

For Recall in Table 2, compared to APHFW, APHF_DTW can increase by 1.64% at the maximum and 0.07% at the minimum. The Recall value is slightly lower than APHFW on Rails and Mybatis, while its value is higher than APHFW on other datasets. On Paint Control, the value increases by 0.07%. On IOF/FOL, the value increases by 1.48%. On GSDTSR, the value increases by 1.64%. On Rails, it decreases by 0.1%. On Mybatis, the value increases by 1.60%. On Apache Drill, the value decreases by 0.20%.

Finally for TTF metric in Table 2, APHF_DTW can decrease by 20.77 at the maximum and 0.24 at the minimum compared to APHFW. The TTF of APHF_DTW is better than APHFW on 5 other datasets except Rails. On Paint Control, the value decreases by 0.24. On IOF/FOL, the value decreases by 0.54. On GSDTSR, the value decreases by 9.92. On Rails, the value increases by 0.15. On Mybatis, the value decreases by 20.77. On Apache Drill, the value decreases by 0.47.

For Rails, the reward function based on dynamic time window does not significantly improve the three evaluation indicators, but the values are very close. In our analysis, the failure rate and frequency of Rails are relatively low, resulting in a more complicated environment, so the dynamic time window based reward function is not effective on Rails.

As can be seen from the above, APHF_DTW is significantly better than APHFW in the distribution of NAPFD, Recall and TTF values.

To further verify the effectiveness of our reward function, we give the NAPFD distribution of the two reward function between different datasets (see Figure 5). From Figure 5, we can find that the overall NAPFD distribution of APHF_DTW on Paint Control and GSDTSR is slightly higher than that of APHFW. However, on IOF/ROL, the advantage of APHF_DTW is more pronounced. The overall NAPFD and median of APHF_DTW are far higher than APHFW. On Rails, there is no difference between the two.

On Mybatis, it can also be seen that the median line and upper edge of the NAPFD distribution of APHF_DTW are significantly higher than that of APHFW. On Apache Drill, the upper edge of the NAPFD distribution of APHF_DTW is significantly higher than that of APHFW. According to the analysis and comparison of the above results, there are five datasets better in APHF_DTW, that is, the distribution of APHF_DTW's NAPFD value is generally higher than that of APHFW on most datasets.

In general, the reward function based on dynamic time window is more effective than the reward function based on fixed time window to prioritize test cases.

RQ3: Does the reward function based on dynamic time window affect the overall efficiency of RL based CI testing in terms of time cost?

RQ3 mainly analyzes whether the time consumption of the reward function based on dynamic time window will affect the overall CI testing efficiency. The TCP process based on RL in CI testing roughly calculates rewards for the agent to learn through the reward function. The agent predicts the priority of test cases and sorts them based on the priority. The sorting process uses a greedy strategy, which has little effect on the overall time consumption. However, the dynamic time window needs to be calculated based on the historical information of the test cases in CI process, which will inevitably cause additional time consumption. We count the total time consumption of a whole CI process including APHFW and APHF_DTW respectively. Table 3 lists the results. It can be seen that because of the calculation of dynamic time window, the time consumption of APHF_DTW is about 1.5s more than APHFW.

Table 3: The comparison of time consumption for APHF_DTW and APHFW reward functions

Dataset	APHFW	APHF_DTW
Paint Control	0.0413	0.4298
IOF/ROL	0.0532	0.2349
GSDTSR	3.6888	11.8186
Rails	0.1529	0.4565
Mybatis	0.2388	0.3888
Apache Drill	0.1672	0.1566
Average	0.7237	2.2475

Table 4 gives the average time window size for two reward function. APHFW's average window size is almost fixed to 5. However,

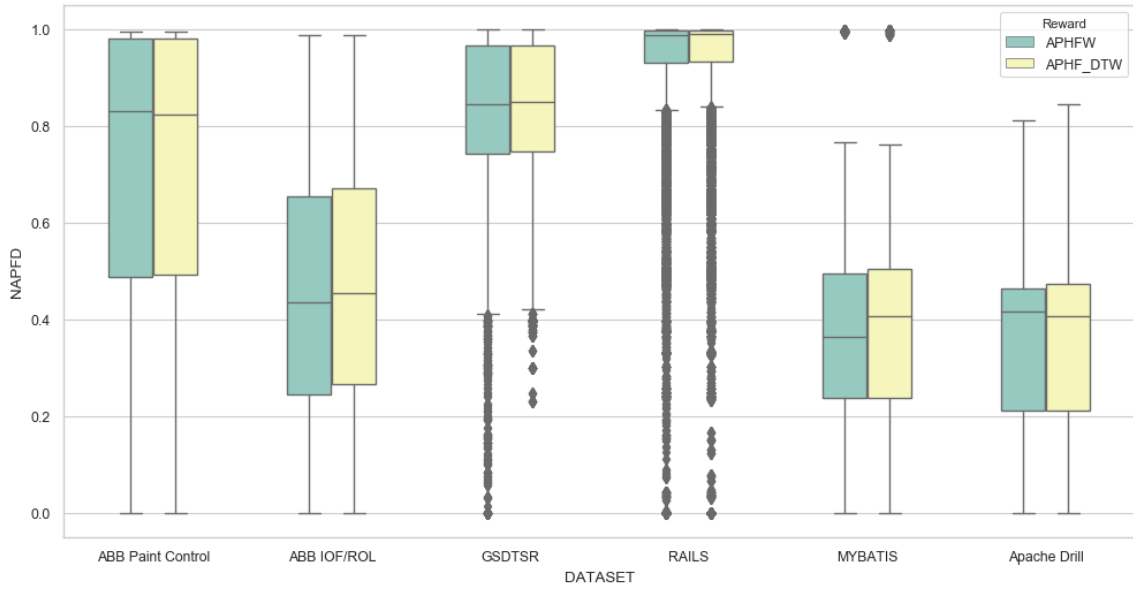


Figure 5: Boxplot of NAPFD Produced by Different Reward Functions between APHF_DTW and APHF

on most datasets, APHF_DTW's average window size is more than 5.00, the average value of all datasets is 8.33, that is, APHF_DTW chooses more effective historical information length than APHF.

Table 4: The comparison of average time window size for APHF_DTW and APHF reward functions

Dataset	APHF	APHF_DTW
Paint Control	5.00	6.77
IOF/ROL	4.94	4.18
GSDTSR	5.00	9.78
Rails	5.00	9.84
Mybatis	5.00	9.62
Apache Drill	4.94	9.77
Average	4.98	8.33

In summary, the reward function based on dynamic time window has increased in time overhead, but the increased range is within seconds, and the longest time does not exceed 9s. However, various indicators such as fault detection rate, recall rate, and TTF have been improved to varying degrees. It selects more effective historical information length. Therefore, for industrial programs, the time increase within seconds is acceptable.

5 THREATS TO VALIDITY

There are mainly three distinct types of validity threats in this context. They are internal validity, external validity and construct validity.

Internal. The agent of RL has the characteristics of random learning in the early exploration stage, which is an internal threat. Therefore, we repeated our experiment 30 times and reported the results on average to avoid this threat.

Research on machine learning has shown that different parameter configurations will lead to different environments. In our experiment, in order to compare with Wu's results [34], we did not adjust the parameters.

It is usually assumed that when the entire test suite can be run and all faults are found, we use APFD's evaluation metric. However, regression testing in CI usually only executes part of the test suite because of its time limit. In fact, these test cases will not contain all errors, so NAPFD is more effective.

External. The six datasets we used are provided by researchers. Three datasets are provided by Spieker [30]. The other three datasets Rails², Mybatis and Apache Drill³ are constructed based on CI testing logs. [1, 16] Additional experiments are needed to verify the performance of dynamic time window in CI testing's TCP problem.

Construct. A threats to construct validity is the assumption, that each failed test case is independent in CI testing. However, that is not always true. The failure of test cases may be related. One test case failure may lead to multiple failures, and vice versa. To address this threat, we suggest to take the features of test cases as input, and based on this, find out test cases similar to failed test cases which has high probability of failure in the future work.

6 CONCLUSION AND FUTURE WORK

Aiming at the RL optimization for CI testing, this paper studies the reward mechanism of RL and the impact of the historical failure interval of the test case execution information in the CI cycle on the fault detection ability of the test sequence, and dynamic time window is defined. Combined with the reward function of the RL method for CI test optimization, an Average Percentage of Historical Failure with Dynamic Time Window reward function

²<http://github.com/elbum/CI-Datasets.git>

³<https://travisreport.testroots.org/buildlogs/>

(APHF_DTW) is proposed. APhF_DTW considers the average historical failure interval of test cases in CI cycle and calculates the dynamic time window to filter out the test case historical information that is out of date for the current cycle. We conduct comparative experiments on 6 large industrial datasets, and the results showed that: (1) Compared with the reward function based on fixed time window, APhF_DTW can better improve the learning efficiency of RL. (2) In terms of fault detection capability, APhF_DTW is more effective to prioritize test cases than APHF. (3) APhF_DTW inevitably increases time consumption, but all are in the second level, which will not have a great impact on the efficiency of CI testing.

In future work, we will focus on: (1) Combining test cases to execute other information features, optimize calculation methods for dynamic time windows, and improve performance on datasets with low failure rates; (2) Combining deep learning algorithms to optimize the agent algorithm of RL to improve the performance of RL in CI testing.

ACKNOWLEDGMENTS

The work described in this paper is supported by the National Natural Science Foundation of China under Grant No.61872026, 61672085, 61702029 and 62077003.

REFERENCES

- [1] Moritz Beller, Georgios Gousios, and Andy Zaidman. 2017. Travistorrent: Synthesizing travis ci and github for full-stack research on continuous integration. In *2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR)*. IEEE, 447–450.
- [2] Yi Bian, Zheng Li, Ruilian Zhao, and Dunwei Gong. 2017. Epistasis based aco for regression test case prioritization. *IEEE Transactions on Emerging Topics in Computational Intelligence* 1, 3 (2017), 213–223.
- [3] Benjamin Busjaeger and Tao Xie. 2016. Learning for Test Prioritization: An Industrial Case Study. In *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering (Seattle, WA, USA) (FSE 2016)*. Association for Computing Machinery, New York, NY, USA, 975–980. <https://doi.org/10.1145/2950290.2983954>
- [4] Benjamin Busjaeger and Tao Xie. 2016. Learning for test prioritization: an industrial case study. In *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering*. 975–980.
- [5] Songyu Chen, Zhenyu Chen, Zhihong Zhao, Baowen Xu, and Yang Feng. 2011. Using semi-supervised clustering to improve regression test selection techniques. In *2011 Fourth IEEE International Conference on Software Testing, Verification and Validation*. 1–10.
- [6] Jianlei Chi, Yu Qu, Qinghua Zheng, Zijiang Yang, Wuxia Jin, Di Cui, and Ting Liu. 2020. Relation-based test case prioritization for regression testing. *Journal of Systems and Software* 163 (2020), 110539.
- [7] Younghwan Cho, Jeongho Kim, and Eunseok Lee. 2016. History-based test case prioritization for failure information. In *2016 23rd Asia-Pacific Software Engineering Conference (APSEC)*. IEEE, 385–388.
- [8] Fujin Deng, Ming Jin, Chengkai Liu, Marco Liserre, and Wu Chen. 2020. Switch Open-Circuit Fault Localization Strategy for MMCs Using Sliding-Time Window Based Features Extraction Algorithm. *IEEE Transactions on Industrial Electronics* (2020).
- [9] Hyunsook Do, Siavash Mirarab, Ladan Tahvildari, and Gregg Rothermel. 2010. The effects of time constraints on test case prioritization: A series of controlled experiments. *IEEE Transactions on Software Engineering* 36, 5 (2010), 593–617.
- [10] Paul M Duvall, Steve Matyas, and Andrew Glover. 2007. *Continuous integration: improving software quality and reducing risk*. Pearson Education.
- [11] Sebastian Elbaum, Gregg Rothermel, and John Penix. 2014. Techniques for improving regression testing in continuous integration development environments. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*. 235–245.
- [12] Alireza Haghighatkah, Mika Mäntylä, Markku Oivo, and Pasi Kuvaja. 2018. Test prioritization in continuous integration environments. *Journal of Systems and Software* 146 (2018), 80–98.
- [13] Dan Hao, Lingming Zhang, Lu Zhang, Gregg Rothermel, and Hong Mei. 2014. A unified test case prioritization approach. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 24, 2 (2014), 1–31.
- [14] Hosney Jahan, Ziliang Feng, SM Mahmud, and Penglin Dong. 2019. Version specific test case prioritization approach based on artificial neural network. *Journal of Intelligent & Fuzzy Systems* 36, 6 (2019), 6181–6194.
- [15] Jeongho Kim, Hohyeon Jeong, and Eunseok Lee. 2017. Failure history data-based test case prioritization for effective regression test. In *Proceedings of the Symposium on Applied Computing*. 1409–1415.
- [16] Jingjing Liang, Sebastian Elbaum, and Gregg Rothermel. 2018. Redefining prioritization: continuous prioritization for continuous integration. In *Proceedings of the 40th International Conference on Software Engineering*. 688–698.
- [17] Zheng, Li Liuliu He, Yang Yang and Ruilian Zhao. 2019. The Reward of Reinforcement learning for Test Optimization for Continuous Integration. *Journal of Software*, 2019, 30(5): 1438-1449(in Chinese) (2019). <http://www.jos.org.cn/1000-9825/5714.htm>
- [18] Congcong Ma, Wenfeng Li, Jingjing Cao, Juan Du, Qimeng Li, and Raffaele Gravina. 2020. Adaptive sliding window based activity recognition for assisted livings. *Information Fusion* 53 (2020), 55–65.
- [19] Mostafa Mahdich, Seyed-Hassan Mirian-Hosseinabadi, Khashayar Etemadi, Ali Nosrati, and Sajad Jalali. 2020. Incorporating fault-proneness estimations into coverage-based test case prioritization methods. *Information and Software Technology* 121 (2020), 106269.
- [20] Duscica Marijan, Arnaud Gotlieb, and Sagar Sen. 2013. Test case prioritization for continuous regression testing: An industrial case study. In *2013 IEEE International Conference on Software Maintenance*. IEEE, 540–543.
- [21] Duscica Marijan and Marius Liaen. 2016. Effect of time window on the performance of continuous regression testing. In *2016 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 568–571.
- [22] Tim Miller et al. 2012. Using dependency structures for prioritization of functional test suites. *IEEE transactions on software engineering* 39, 2 (2012), 258–275.
- [23] Ali Parsai, Quinten David Soetens, Alessandro Murgia, and Serge Demeyer. 2014. Considering polymorphism in change-based test suite reduction. In *International Conference on Agile Software Development*. Springer, 166–181.
- [24] Pilar Rodriguez, Alireza Haghighatkah, Lucy Ellen Lwakatare, Susanna Teppola, Tanja Suomalainen, Juho Eskeli, Teemu Karvonen, Pasi Kuvaja, June M Verner, and Markku Oivo. 2017. Continuous deployment of software intensive products and services: A systematic mapping study. *Journal of Systems and Software* 123 (2017), 263–291.
- [25] Gregg Rothermel and Mary Jean Harrold. 1997. A safe, efficient regression test selection technique. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 6, 2 (1997), 173–210.
- [26] Gregg Rothermel, Roland H. Untch, Chengyun Chu, and Mary Jean Harrold. 2001. Prioritizing test cases for regression testing. *IEEE Transactions on software engineering* 27, 10 (2001), 929–948.
- [27] Saijun Shao, Gangyan Xu, Ming Li, and George Q Huang. 2019. Synchronizing e-commerce city logistics with sliding time windows. *Transportation Research Part E: Logistics and Transportation Review* 123 (2019), 17–28.
- [28] August Shi, Suresh Thummalapenta, Shuvendu K Lahiri, Nikolaj Björner, and Jacek Czerwinka. 2017. Optimizing test placement for module-level regression testing. In *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*. IEEE, 689–699.
- [29] Quinten David Soetens, Serge Demeyer, and Andy Zaidman. 2013. Change-based test selection in the presence of developer tests. In *2013 17th European Conference on Software Maintenance and Reengineering*. IEEE, 101–110.
- [30] Helge Spieker, Arnaud Gotlieb, Duscica Marijan, and Morten Mossige. 2018. Reinforcement learning for automatic test case prioritization and selection in continuous integration. *arXiv preprint arXiv:1811.04122* (2018).
- [31] Hema Srikanth, Charitha Hettiarachchi, and Hyunsook Do. 2016. Requirements based test prioritization using risk factors: An industrial study. *Information and Software Technology* 69 (2016), 71–83.
- [32] Bogdan Vasilescu, Yue Yu, Huaimin Wang, Premkumar Devanbu, and Vladimir Filkov. 2015. Quality and productivity outcomes relating to continuous integration in GitHub. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*. 805–816.
- [33] W Eric Wong, Joseph R Horgan, Saul London, and Hiralal Agrawal. 1997. A study of effective regression testing in practice. In *PROCEEDINGS The Eighth International Symposium On Software Reliability Engineering*. IEEE, 264–274.
- [34] Zhaolin Wu, Yang Yang, Zheng Li, and Ruilian Zhao. 2019. A Time Window based Reinforcement Learning Reward for Test Case Prioritization in Continuous Integration. In *Proceedings of the 11th Asia-Pacific Symposium on Internetware*. 1–6.
- [35] Yang Yang, Zheng Li, Liuliu He, and Ruilian Zhao. 2020. A systematic study of reward for reinforcement learning based continuous integration testing. *Journal of Systems and Software* 170 (2020), 110787.
- [36] Lian Yu, Lei Xu, and Wei-Tek Tsai. 2010. Time-constrained test selection for regression testing. In *International Conference on Advanced Data Mining and Applications*. Springer, 221–232.